

Sentinel Lab — Neuro-Symbolic Explainable Cyber Threat Detection

Full Build Specification for Autonomous / Semi-Autonomous Coding Agents

Project: A Neuro-Symbolic Architecture for Explainable Reasoning in Deep Learning Systems

Prepared September 2026

Contents

1	0. How to Use This Document	3
2	1. Project Summary	4
3	2. System Architecture	5
3.1	2.1 High-level data flow	5
3.2	2.2 Component list	5
3.3	2.3 Repository layout	7
4	3. Neural Classifier	9
4.1	3.1 Datasets	9
4.2	3.2 Feature extraction	9
4.3	3.3 Model architecture (start here, iterate later)	9
4.4	3.4 Inference server	10
5	4. Symbolic Reasoning & Knowledge Base	11
5.1	4.1 Knowledge base content	11
5.2	4.2 Reasoning engine	11
6	5. Faithfulness Verification (the project's key contribution)	12
6.1	5.1 What "faithful" means here	12
6.2	5.2 Method	12
6.3	5.3 Reasoning-shortcut awareness	12
7	6. Build Phases & Milestones	13
7.1	Phase 0 — Repo & environment scaffolding	13
7.2	Phase 1 — Data & classifier	13
7.3	Phase 2 — Knowledge base & reasoning engine	13
7.4	Phase 3 — Faithfulness verification	13
7.5	Phase 4 — Explanation generation + event pipeline	13
7.6	Phase 5 — Lab infrastructure & live traffic (see companion homelab document)	14
7.7	Phase 6 — Dashboard	14
7.8	Phase 7 — Adversarial testing & evaluation	14
8	7. XAI Dashboard	15
8.1	7.1 API surface (event-api service)	15
8.2	7.2 UI layout (reference mockup)	15
9	8. Event Database Schema (starting point)	16

109. Attack Simulator (lab-only)	17
11 10. Evaluation Plan	18
12 11. Suggested Build Order for a Multi-Agent Harness	19
13 12. References (carried over from the literature survey)	20

1 0. How to Use This Document

This document is written to be handed directly to a coding agent / harness (Claude Code, DeepSeek Harness, or similar) as a build specification. It is organized so the agent can work **phase by phase, module by module**, without needing the whole system understood in one context window.

Rules for the agent building this:

1. Never build the whole system in a single pass. Follow the phases in order (Section 6). Each phase ends with a milestone check — do not proceed until it passes.
2. Treat the *detection/blocking* path and the *explanation* path as separate concerns. Deterministic rules (WAF, network policy) block traffic. The neural + symbolic layer explains *why*, it does not gate production traffic.
3. Every vulnerable/demo application lives only inside the isolated lab namespace/network described in the companion document (“Two-Node Xeon Kubernetes Homelab Setup Guide”). Nothing in this project is exposed to the public internet.
4. Prefer small, testable increments with a working end-to-end path over a feature-complete-but-untested module. A thin vertical slice (one request → one blocked/allowed decision → one stored event → one generated explanation → one dashboard row) should exist before any component is deepened.
5. Log architectural decisions and deviations from this spec in docs/decisions/ADR-<n>-<slug>.md (lightweight Architecture Decision Records) as you go.

2 1. Project Summary

Title: A Neuro-Symbolic Architecture for Explainable Reasoning in Deep Learning Systems for Cyber Threat Detection

Problem: Deep-learning intrusion/threat detection achieves strong accuracy but functions as an opaque black box. Analysts get a threat score or a SHAP heatmap, not a logically valid, auditable justification tied to known attack patterns (e.g., MITRE ATT&CK). Existing neuro-symbolic IDS work generates a symbolic rule trace but rarely verifies that the trace actually reflects the model’s internal computation (“faithfulness”). Attackers can exploit degenerate neural mappings (reasoning shortcuts) to evade detection while superficially passing rule checks.

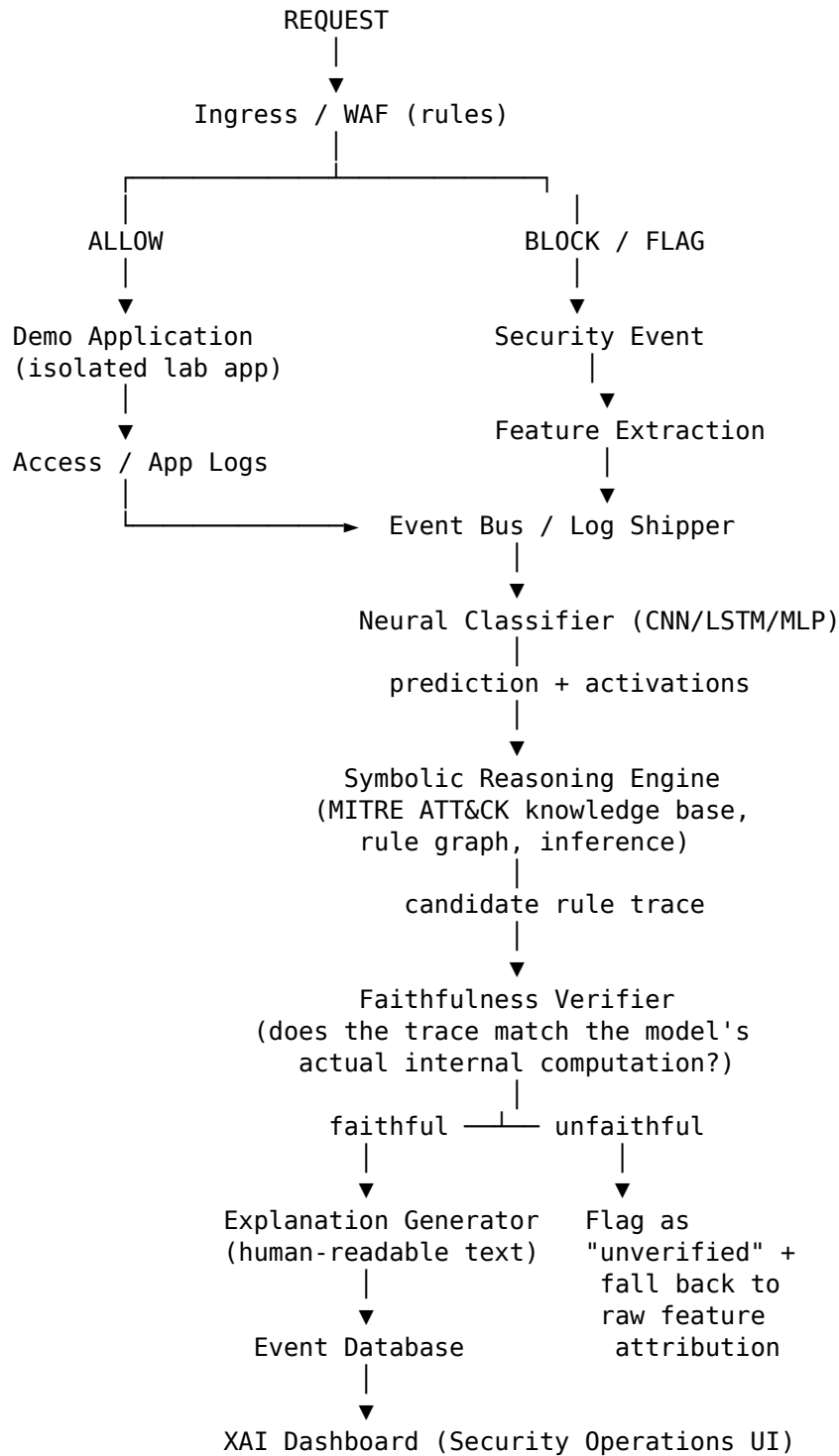
Objectives:

1. Build a neuro-symbolic cyber threat detection framework combining a deep-learning classifier with a symbolic reasoning engine.
2. Build a cybersecurity knowledge base of expert-defined attack rules/relationships grounded in MITRE ATT&CK.
3. Generate human-readable, auditable explanations that connect the neural prediction to the symbolic rules that fired.
4. Verify that the generated explanation is **faithful** to the model’s actual internal decision process, not a post-hoc rationalization.
5. Evaluate on accuracy, precision, recall, F1, false-positive rate, rule validity, and explanation faithfulness.

Design principle (critical): the deterministic security controls (WAF rules, network policy, signature match) are what block traffic. The neural+symbolic layer’s job is to explain and classify with an audit trail — it is not the sole gate for blocking production-style traffic. This keeps the system safe, debuggable, and evaluable, and it directly targets objective 4 (faithfulness) which the literature review found is the field’s biggest open gap.

3 2. System Architecture

3.1 2.1 High-level data flow



3.2 2.2 Component list

#	Component	Responsibility	Suggested tech
1	Ingress / WAF	Deterministically allow/block requests using signature + OWASP CRS rules	Traefik or NGINX ingress + Coraza/ModSecurity, OWASP CRS
2	Demo applications	Intentionally vulnerable practice targets that generate realistic attack + benign traffic	OWASP Juice Shop, DVWA, WebGoat, one custom CRUD app
3	Telemetry collector	Ship WAF logs, app logs, network flow logs to a common pipeline	Fluent Bit / Vector → Kafka or Redis Streams (or direct to Postgres for v1)
4	Feature extraction service	Turn raw logs/flows into the statistical/behavioral feature vectors the model expects	Python service, pandas/numpy, scikit-learn preprocessing
5	Neural classifier	Learn attack vs. benign (and attack-type) patterns from flow features	PyTorch: 1D-CNN + MLP head (start simple, see §3)
6	Knowledge base	Store attack rules, MITRE ATT&CK technique mappings, feature→rule relationships	Rules as YAML/JSON + a lightweight rule engine (see §4.2); optionally a graph DB (Neo4j) for the technique graph
7	Symbolic reasoning engine	Take the model’s prediction + input features, run inference against the KB, produce a candidate logical trace	Python rule engine (experta/custom forward-chaining) or Prolog-style engine (pyDatalog)
8	Faithfulness verifier	Check whether the symbolic trace is actually grounded in the features the neural model used (via saliency/activation alignment)	Captum (integrated gradients / saliency) + custom scoring, inspired by BEARS / ActivationReasoning (see §5)
9	Explanation generator	Render the verified trace into a human-readable narrative	Template-based NLG first; optional LLM polish later behind a strict grounding check
10	Event database	Durable store of every event: raw features, prediction, trace, faithfulness score, explanation	PostgreSQL

#	Component	Responsibility	Suggested tech
11	XAI dashboard	Security Operations Center style UI: today's stats, event list, click-through explanation view	React + a small Go/Python API (see §7)
12	Attack simulator	Generate labeled attack + benign traffic against the demo apps, only inside the lab	sqlmap, OWASP ZAP, custom scripts — controlled and scheduled, lab-only
13	Evaluation harness	Compute accuracy/precision/recall/F1/FPR + rule validity + faithfulness score against held-out data	Python, scikit-learn, FRERs, custom faithfulness metric

3.3 2.3 Repository layout

```

sentinel-lab/
├── apps/                                # intentionally-vulnerable demo targets (isolated only)
│   ├── demo-shop/                      # e.g., wraps/deploys Juice Shop
│   ├── demo-blog/                      # simple custom CRUD app (own code, seeded bugs)
│   └── demo-bank/                      # optional, higher-stakes demo target
├── infrastructure/
│   ├── kubernetes/                    # namespaces, deployments, services
│   ├── networking/                   # NetworkPolicies, Cilium policies
│   ├── storage/                      # PVCs, StorageClasses
│   └── ingress/                      # Traefik/NGINX + WAF config
├── security/
│   ├── waf/                          # OWASP CRS rule sets, custom rules
│   ├── attack-simulator/             # scripted attack scenarios (sqlmap/ZAP configs)
│   ├── policies/                     # NetworkPolicy definitions per app
│   └── detection/                    # signature rules feeding the event pipeline
├── services/
│   ├── ingest/                       # log/flow shipping + normalization
│   ├── feature-extraction/           # raw log -> feature vector
│   ├── classifier/                   # neural model training + inference server
│   ├── knowledge-base/               # rule definitions + MITRE ATT&CK mapping data
│   ├── reasoning-engine/             # symbolic inference over KB
│   ├── faithfulness/                 # faithfulness scoring service
│   ├── explain/                      # explanation generator (NLG)
│   └── event-api/                    # REST/GraphQL API over the event database
├── dashboard/                        # React XAI dashboard (frontend)
├── database/
│   ├── migrations/
│   └── schema.sql
└── notebooks/                        # exploratory training/evaluation notebooks

```

```
|
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── security/
│   └── attack-scenarios/
│
└── docs/
    ├── architecture/
    ├── threat-model/
    ├── decisions/
    └── runbooks/
```

WAF/network-policy verification tests
end-to-end: attack -> detect -> explain

ADRs

4 3. Neural Classifier

4.1 3.1 Datasets

Use public, well-documented network intrusion datasets. Do not attempt to source real production traffic.

Dataset	Use for	Notes
CICIDS2017	Primary training set	Balanced mix of attack types, well-labeled
CSE-CIC-IDS2018 UNSW-NB15	Secondary/cross-validation set Generalization test	Larger, more attack diversity Different feature schema — good for testing robustness
NF-BoT-IoT-V2	Optional IoT-flavored extension	Matches the closest related work (Almadhor et al., 2025)

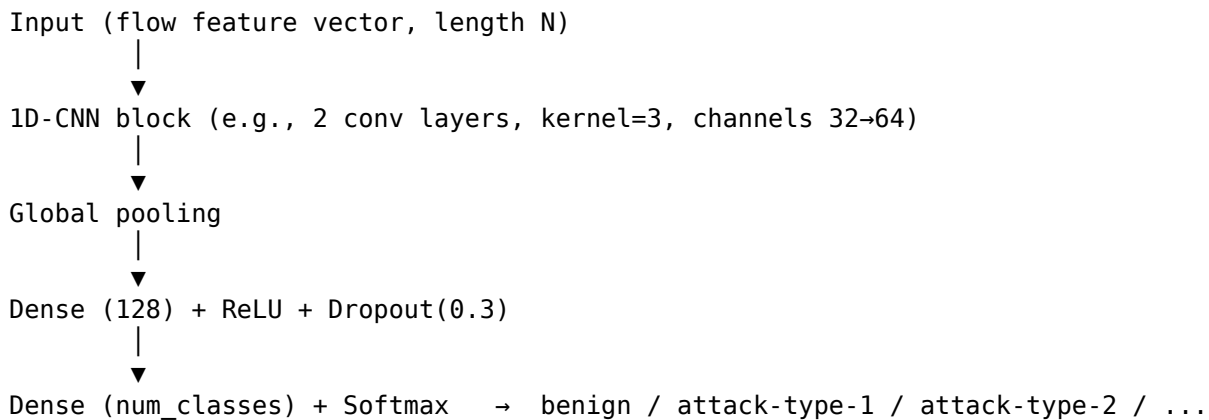
Pipeline: download → clean (drop NaN/duplicate flows) → encode categoricals → normalize numeric features → stratified train/val/test split (e.g., 70/15/15) → persist processed parquet files under `services/feature-extraction/data/processed/`.

4.2 3.2 Feature extraction

- Extract standard flow-based statistical features (duration, byte/packet counts, inter-arrival times, flag counts, protocol) plus a small set of behavioral features (request rate, unique endpoints hit, payload entropy) for the lab’s own HTTP-layer traffic.
- Represent each flow as a fixed-length numeric vector; keep a `feature_names.json` manifest so every downstream component (classifier, symbolic engine, faithfulness verifier) references features by stable name, not index.
- Generate compact embeddings for high-cardinality categorical fields (e.g., destination endpoint) rather than one-hot blowing up the input width.

4.3 3.3 Model architecture (start here, iterate later)

Start with a small, well-understood architecture — the goal of this project is explainability, not squeezing out the last 0.1% accuracy.



- Frame as multi-class classification (benign + one class per attack category present in the dataset), not just binary, so the symbolic layer has something specific to reason about (e.g., “classified as SQL_INJECTION”).

- Log training runs (metrics, hyperparameters, dataset version) — a simple `services/classifier/experiments` folder with one JSON per run is sufficient for a course project; MLflow is a reasonable upgrade if the agent has time budget.
- Export the trained model in a format the inference server and the faithfulness verifier can both load (e.g., TorchScript, or plain `state_dict` + the same model class).

4.4 3.4 Inference server

- Small FastAPI (or Flask) service: `POST /predict` takes a feature vector, returns `{predicted_class, confidence, per_class_probs, activation_snapshot}`.
- `activation_snapshot` should include the intermediate layer activations / gradients needed later by the faithfulness verifier (e.g., via a forward hook). Don't bolt this on later — wire the hook in from the first working version of the server.

5 4. Symbolic Reasoning & Knowledge Base

5.1 4.1 Knowledge base content

Model the knowledge base as **rules that connect observable features to MITRE ATT&CK techniques**, not as a black-box lookup. Each rule should be independently auditable by a human analyst.

Example rule record (services/knowledge-base/rules/sqli.yaml):

```
rule_id: R-SQLI-001
name: "SQL Injection via query parameter"
attack_type: SQL_INJECTION
mitre_technique: "T1190"      # Exploit Public-Facing Application
mitre_tactic: "TA0001"       # Initial Access
conditions:
  - feature: payload_sql_keyword_count
    operator: ">="
    value: 1
  - feature: payload_special_char_ratio
    operator: ">"
    value: 0.15
  - feature: http_method
    operator: "in"
    value: ["GET", "POST"]
severity: high
explanation_template: >
  The request contained {payload_sql_keyword_count} SQL-syntax keyword(s) and
  an elevated ratio of special characters ({payload_special_char_ratio}),
  consistent with an attempt to alter the application's SQL query logic
  (MITRE ATT&CK T1190).
```

Seed the knowledge base with rules covering at minimum: SQL injection, XSS, path traversal, command injection, brute-force/credential stuffing, and one anomaly-only category (traffic that is “suspicious” but does not match any known rule — this matters for the evaluation of rule *validity/coverage*).

5.2 4.2 Reasoning engine

- Implement as **forward-chaining rule evaluation**: given the feature vector and the neural model’s predicted class, evaluate every rule whose `attack_type` matches (or is a candidate for) the predicted class, and collect all rules whose conditions are satisfied.
- Keep this engine pure and side-effect free: `evaluate(features, predicted_class) -> List[FiredRule]`. This makes it independently unit-testable against fixture feature vectors, separate from the neural model.
- A lightweight hand-rolled Python engine is entirely adequate for this project’s scope; only reach for pyDatalog/experta/Prolog-style tooling if the rule graph grows complex enough to need real backward-chaining or contradiction detection.
- Where more than one rule fires, rank by severity and specificity (more conditions matched = more specific) and surface the top-ranked rule(s) to the explanation generator.

6 5. Faithfulness Verification (the project’s key contribution)

This is what differentiates this project from the existing neuro-symbolic IDS literature reviewed (which largely does not verify that the symbolic trace reflects the model’s actual computation).

6.1 5.1 What “faithful” means here

A symbolic explanation is faithful if the features it cites as evidence are the same features the neural model’s internal computation actually relied on for that prediction — not merely features that happen to correlate with the predicted label in general.

6.2 5.2 Method

1. For a given prediction, compute a feature-importance attribution from the neural model itself (integrated gradients or saliency via Captum, or a simpler gradient×input approach for v1).
2. Compare the *top-k most important features by attribution* against the *features cited by the fired symbolic rule(s)*.
3. Compute an overlap/agreement score (e.g., a weighted Jaccard between the two feature sets, or rank-correlation if attribution provides an ordering).
4. Define a threshold (tune empirically on a held-out set) above which the explanation is marked **faithful**, and below which it is marked **unverified** — in the unverified case, do not present the symbolic narrative as-is; fall back to presenting the raw top attributed features with a clear “not rule-verified” label.
5. Log the faithfulness score with every event, and report the **overall faithfulness rate** as one of the project’s headline evaluation metrics (this is explicitly what objective 4 and the evaluation criteria ask for).

6.3 5.3 Reasoning-shortcut awareness

Borrow the framing from Marconato et al. (BEARS, 2024): a model can produce the “right” label for the “wrong” reason (a reasoning shortcut). During evaluation, deliberately construct a small set of adversarial/edge-case flows designed to trigger this (e.g., benign traffic that happens to contain SQL-like keywords in a non-exploitable context) and check whether the faithfulness verifier correctly flags the resulting explanation as unverified rather than letting a spurious rule match through.

7 6. Build Phases & Milestones

Build in this order. Each phase has an explicit “done” condition — do not start the next phase until it is met.

7.1 Phase 0 — Repo & environment scaffolding

- Create the repository layout from §2.3.
- Set up Python virtualenv / poetry for the ML services, and a separate Node/React project for the dashboard.
- Set up a local docker-compose for fast iteration on services *before* anything is deployed to Kubernetes (the K8s cluster is described in the companion homelab document).
- **Done when:** docker-compose up brings up an empty Postgres + a “hello world” FastAPI service that can write and read one row.

7.2 Phase 1 — Data & classifier

- Implement dataset download/clean/feature-extraction pipeline (§3.1–3.2).
- Train the baseline 1D-CNN classifier (§3.3) on CICIDS2017.
- Stand up the inference server (§3.4) with activation-snapshot hooks wired in from day one.
- **Done when:** POST /predict on the inference server returns a class, confidence, and activation snapshot for a sample flow, and offline evaluation on the held-out test split reports accuracy/precision/recall/F1/FPR.

7.3 Phase 2 — Knowledge base & reasoning engine

- Author the initial rule set (§4.1) covering the attack categories present in the training data.
- Implement the reasoning engine (§4.2) as a pure function with unit tests against fixture feature vectors (both rules that should fire and rules that should not).
- **Done when:** given a labeled test flow and its predicted class, the engine returns the expected fired rule(s) for at least 90% of a hand-checked sample set.

7.4 Phase 3 — Faithfulness verification

- Implement attribution extraction (Captum) against the trained model.
- Implement the overlap scoring and threshold tuning (§5.2).
- Build the adversarial/reasoning-shortcut test set (§5.3) and confirm the verifier flags them as expected.
- **Done when:** the faithfulness rate metric is computed end-to-end and the shortcut test set is correctly flagged as “unverified” more often than the clean test set.

7.5 Phase 4 — Explanation generation + event pipeline

- Implement the template-based explanation generator, consuming the fired rule + faithfulness verdict.
- Stand up the Postgres event schema (§8) and the event-api service.
- Wire ingest → feature-extraction → classifier → reasoning-engine → faithfulness → explain → event-api as one pipeline (can be a single orchestrating service for v1; split into microservices only if the harness has time budget).
- **Done when:** a single log line pushed into ingest results in a fully-populated row in the event database, including a human-readable explanation string.

7.6 Phase 5 — Lab infrastructure & live traffic (see companion homelab document)

- Deploy the WAF/ingress, demo applications, and network policies to the Kubernetes lab.
- Wire the WAF/app logs into the ingest service from Phase 4.
- **Done when:** a manually-issued benign request and a manually-issued attack request (e.g., a single SQLi test string against the demo shop, run only against your own isolated lab instance) each produce a correctly-labeled, explained event end-to-end.

7.7 Phase 6 — Dashboard

- Build the React XAI dashboard (§7): stats header, event list, click-through detail view matching the mockup in §7.2.
- **Done when:** the dashboard shows live events as they land in the event database and clicking an event shows the full explanation, faithfulness verdict, and MITRE technique.

7.8 Phase 7 — Adversarial testing & evaluation

- Run the attack simulator (§9) against the lab to generate a larger labeled corpus of real (lab) traffic.
- Compute final evaluation numbers (§10) on this corpus and compare against the offline dataset numbers from Phase 1.
- Write up results, including any cases where symbolic explanations were flagged unfaithful, in docs/architecture/evaluation-results.md.
- **Done when:** the full evaluation table (accuracy, precision, recall, F1, FPR, rule validity, faithfulness rate) is populated with real numbers from both the offline dataset and the live lab traffic.

8 7. XAI Dashboard

8.1 7.1 API surface (event-api service)

Method	Path	Description
GET	/api/v1/stats/today	Counts: attacks today, blocked, suspicious
GET	/api/v1/events?limit=&type=&page=	Pageinated recent events
GET	/api/v1/events/{id}	Full detail: features, prediction, fired rules, faithfulness score, explanation, evidence
GET	/api/v1/metrics	Rolling accuracy/precision/recall/F1/FPR/faithfulness rate

8.2 7.2 UI layout (reference mockup)

SECURITY OPERATIONS	
Attacks Today	147
Blocked	139
Suspicious	8
Recent Events	
SQL Injection	BLOCKED
XSS	BLOCKED
Path Traversal	BLOCKED
Brute Force	MITIGATED

Click an event ->

SQL INJECTION

What happened?	The request contained a pattern consistent with SQL injection.
Why was it flagged?	Rule R-SQLI-001 fired; faithfulness score 0.87 (verified).
Mitre mapping	T1190 / TA0001
Target	/api/login
Evidence	[feature snapshot] [rule trace] [attribution chart]

- Use a small charting library (recharts/chart.js) for the attribution chart and the faithfulness-score trend.
- Explicitly render the faithfulness verdict (verified / unverified) — this is the project's differentiator, do not bury it in a tooltip.

9 8. Event Database Schema (starting point)

```
CREATE TABLE events (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ts          TIMESTAMPTZ NOT NULL DEFAULT now(),  
  source_ip   INET,  
  target_endpoint TEXT,  
  raw_features JSONB NOT NULL,  
  predicted_class TEXT NOT NULL,  
  confidence  DOUBLE PRECISION NOT NULL,  
  fired_rule_ids TEXT[] NOT NULL DEFAULT '{}',  
  mitre_technique TEXT,  
  mitre_tactic TEXT,  
  faithfulness_score DOUBLE PRECISION,  
  faithfulness_verdict TEXT CHECK (faithfulness_verdict IN ('verified', 'unverified', 'not_applied')),  
  explanation_text TEXT,  
  action_taken TEXT CHECK (action_taken IN ('BLOCKED', 'ALLOWED', 'MITIGATED', 'FLAGGED')),  
  ground_truth_label TEXT -- populated only for lab/simulated traffic  
);  
  
CREATE INDEX idx_events_ts ON events (ts DESC);  
CREATE INDEX idx_events_class ON events (predicted_class);
```

10 9. Attack Simulator (lab-only)

- Runs **only** against the demo applications inside the isolated lab namespace/network described in the companion homelab document — never against anything reachable from the public internet.
- Use established, well-documented security-testing tools rather than hand-rolled exploit code: **OWASP ZAP** for automated scanning, **sqlmap** for SQL-injection test scenarios against the intentionally-vulnerable demo apps, and simple scripted HTTP clients for brute-force/credential-stuffing scenarios.
- Tag every simulated request with a ground-truth label (attack type or “benign”) so it can be written to `ground_truth_label` in the event table — this labeled lab traffic is what lets you compute real (not just offline-dataset) precision/recall numbers in Phase 7.
- Schedule simulator runs (a simple cron-style job or a manual `make attack-run TYPE=sqli target`) rather than running continuously, so the event database has clean, reviewable batches.

11 10. Evaluation Plan

Metric	Computed from	Target framing
Accuracy / Precision / Recall / F1	Offline test split (Phase 1) and live lab traffic (Phase 7)	Standard classification metrics per attack class + macro-average
False-positive rate	Same	Benign flows incorrectly classified as attack
Rule validity / coverage	Symbolic engine unit tests + live traffic	% of attack events for which at least one rule fired; % of rules that have fired at least once (dead-rule detection)
Faithfulness rate	Faithfulness verifier (§5)	% of fired-rule explanations whose attribution-overlap score exceeds the tuned threshold
Shortcut robustness	Adversarial test set (§5.3)	Faithfulness verifier correctly down-scores explanations on the adversarial set relative to the clean set

Report all of these in docs/architecture/evaluation-results.md, alongside a short discussion of any dataset limitations (e.g., single dataset family, synthetic lab traffic vs. real-world traffic) — this maps directly to the “Cons” column the literature survey identified in comparable published work.

12 11. Suggested Build Order for a Multi-Agent Harness

If your harness (DeepSeek Harness, Claude Code, or similar) supports subagent delegation, split work along these lines so no single agent context has to hold the whole system:

Orchestrator

— Data/ML agent	→ Phase 1 (dataset, classifier, inference server)
— Symbolic agent	→ Phase 2 (knowledge base, reasoning engine)
— XAI agent	→ Phase 3 (faithfulness verifier)
— Backend agent	→ Phase 4 (event pipeline, event-api, DB schema)
— Infra agent	→ Phase 5 (K8s deployment – see companion doc)
— Frontend agent	→ Phase 6 (dashboard)
— QA/eval agent	→ Phase 7 (attack simulator, evaluation)

Route routine, well-specified implementation work (Kubernetes manifests, CRUD boilerplate, test fixtures, documentation) to whichever cheaper/faster model your harness has available, and reserve your most capable model for: the faithfulness-verification design (§5), the reasoning-engine rule design (§4), and reviewing agent-generated code before it merges. Give agents access only to the repo and a dev cluster/namespace — never home-network-wide or cluster-admin credentials — and gate any change that touches infrastructure/ or security/ behind a human review step.

13 12. References (carried over from the literature survey)

1. Neurosymbolic learning and domain knowledge-driven explainable AI for enhanced IoT network attack detection and response, *Computers & Security*, 2025.
2. Neuro-Symbolic AI for Cybersecurity: State of the Art, Challenges, and Opportunities, arXiv, 2025-26.
3. Neurosymbolic AI in Cybersecurity: Bridging Pattern Recognition and Symbolic Reasoning, IEEE, 2023.
4. Towards Cognitive AI Systems: Workload and Characterization of Neuro-Symbolic AI, IEEE, 2024.
5. Colelough & Regli, Neuro-Symbolic AI in 2024: A Systematic Review, 2025.
6. Marconato et al., BEARS Make Neuro-Symbolic Models Aware of their Reasoning Shortcuts, 2024.
7. A Neuro-Symbolic Benchmark Suite for Concept Quality and Reasoning Shortcuts, arXiv, 2025.
8. ActivationReasoning: Logical Reasoning in Latent Activation Spaces, arXiv, 2025.
9. ExAI5G: Logic-Based XAI Framework for 5G IDS, arXiv, 2026.
10. Labeling NIDS Rules with MITRE ATT&CK: ML vs. LLMs, MDPI, 2025.